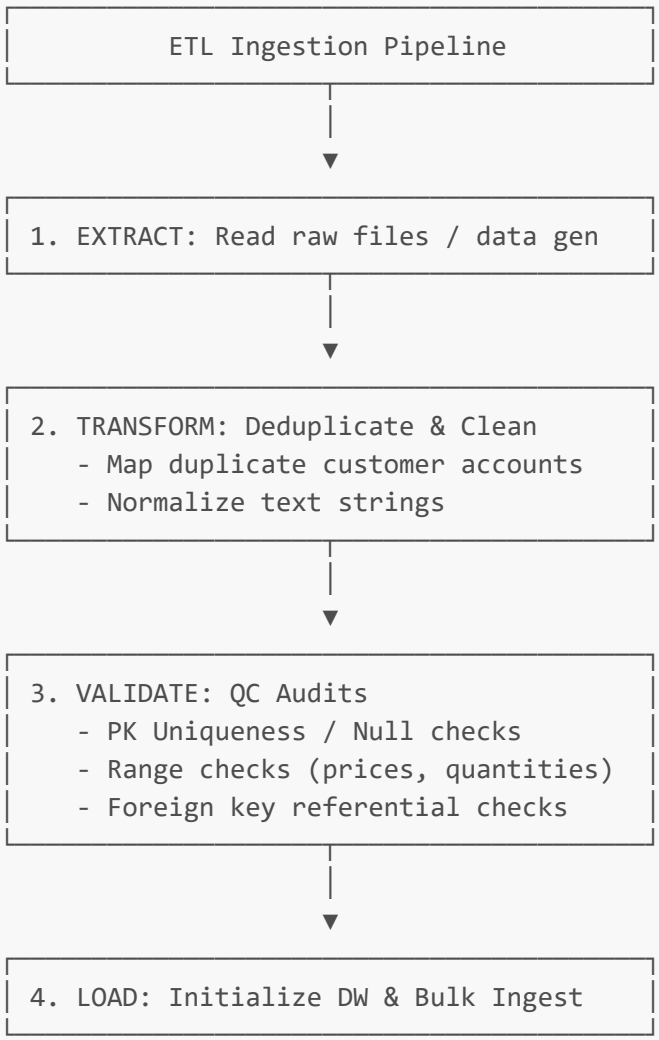
DecisionIQ Master Handbook

CHAPTER 3: Ingestion, Validation, and ETL Pipelines

3.1 The Ingestion Architecture

A data warehouse is only as good as the quality of the data flowing into it. If dirty, corrupt, or duplicate data is loaded, downstream reports will display incorrect information, leading to poor business decisions. This is the classic **"Garbage In, Garbage Out"** problem.

To solve this, DecisionIQ builds a three-stage **ETL (Extract, Transform, Load)** pipeline in Python:



3.2 Master Data Management: Account Consolidation

In enterprise sales, customer records frequently get duplicated. For example, a customer might register twice using different names or IDs but the same email address:

- `CUST00042`: John Doe | `john.doe@company.com`

- `CUST00089`: J. Doe | `john.doe@company.com`

If the ETL pipeline simply drops the duplicate customer record `CUST00089` from the database, it will crash when trying to load transactions referencing the deleted customer ID, throwing a **Foreign Key constraint violation**.

3.3.1 The Account Re-Keying Strategy

To prevent orphan transactions, our ETL pipeline implements an automated **account consolidation mapping** before loading the tables:

1. **Identify Duplicates:** Sort customers by `acquisition_date`. Identify duplicates by scanning for duplicate `contact_email` addresses, keeping the earliest record as the "Golden Master."
2. **Generate ID Mapping:** Extract a mapping dictionary linking the duplicate customer IDs to their master customer ID:

```
# Identify duplicate records
is_duplicate = customers_raw.duplicated(subset=["contact_email"],
keep="first")

# Extract master records
master_ids = customers_raw[~is_duplicate][["contact_email",
"customer_id"]].rename(
    columns={"customer_id": "master_customer_id"}
)

# Extract duplicate records
duplicate_ids = customers_raw[is_duplicate][["contact_email", "customer_id"]]

# Merge and build the mapping dictionary
mapping_df = duplicate_ids.merge(master_ids, on="contact_email", how="left")
customer_id_map = dict(zip(mapping_df["customer_id"],
mapping_df["master_customer_id"]))
```

3. **Re-key Child Records:** Replace references to the duplicate customer ID with the master customer ID in all child tables (`orders` and `customer_support`) before loading them:

```
if customer_id_map:
    orders_df["customer_id"] =
orders_df["customer_id"].replace(customer_id_map)
    support_df["customer_id"] =
support_df["customer_id"].replace(customer_id_map)
```

4. **Deduplicate Customer Dimension:** Drop the duplicate customer records from the `customers` dimension table.

This ensures all child records maintain perfect referential integrity with the cleaned master customers table, preserving transaction history without database integrity crashes.

3.3 Quality Audits & Range Constraints

The `validator.py` script acts as a quality gatekeeper. It runs three core programmatic checks:

1. Primary Key Uniqueness & Null Check

Verifies that the primary key column for each table contains no duplicates and zero null values.

```
null_count = df[pk_col].isnull().sum()
dup_count = df[pk_col].duplicated().sum()
if null_count > 0 or dup_count > 0:
    raise ValueError(f"Table '{table_name}' failed PK audit.")
```

2. Value Range Audits

Enforces logical business bounds on numerical columns (e.g., product prices, order quantities, and support scores cannot be negative):

```
# Check that product prices are positive
if (products_df["unit_price"] < 0).sum() > 0:
    raise ValueError("Failed Range Audit: Negative product prices detected.")
```

3. Referential Integrity (Foreign Key) Audits

Ensures that all values in child table foreign key columns exist in parent table primary key columns.

```
child_vals = df_child[fk_col].dropna().unique()
parent_vals = set(df_parent[pk_col].unique())
invalid_refs = [val for val in child_vals if val not in parent_vals]
if len(invalid_refs) > 0:
    raise ValueError(f"Referential integrity failure: {invalid_refs}")
```

Only when all validation audits pass does the ETL pipeline proceed to load the data warehouse.